

Implementación del Data Warehouse y procesos ETL

2.1 Introducción

En esta fase se desarrollan e implementan los procesos ETL necesarios para poblar el Data Warehouse diseñado en la fase anterior. Cada proceso ETL se ha implementado mediante un script independiente en Python, permitiendo una arquitectura modular, reproducible y fácilmente mantenible.

Para cada script se describen de forma diferenciada las fases de extracción, transformación y carga, siguiendo la metodología clásica ETL aplicada en proyectos de Business Intelligence.

2.2 ETL de la dimensión tiempo (load_dim_date.py)

2.2.1 Extracción.

La dimensión tiempo no procede directamente de un único fichero CSV, sino que se genera a partir de los rangos de fechas presentes en los datos operacionales (pedidos). Se define un intervalo temporal completo que cubre todas las fechas relevantes del dataset.

2.2.2 Transformación

Durante esta fase se generan atributos derivados a partir de cada fecha:

- Fecha completa
- Año
- Mes
- Día
- Trimestre
- Semana del año
- Día de la semana

Además, se construye una clave sustituta `date_sk` en formato YYYYMMDD, utilizada posteriormente como clave foránea en las tablas de hechos.

2.2.3 Carga

Los registros generados se insertan en la tabla `dim_date`, creando una dimensión temporal completa y preparada para análisis agregados por distintos niveles de granularidad temporal.

2.3 ETL de la dimensión cliente (load_dim_customer.py)

2.3.1 Extracción

Los datos se extraen del fichero `olist_customer_dataset.csv`, que contiene información de clientes asociada a los pedidos realizados en la plataforma.

2.3.2 Transformación

Durante la transformación se realizan las siguientes operaciones:

- Conversión y limpieza de campos de texto.
- Deduplicación de clientes utilizando el atributo `customer_unique_id`, ya que un mismo cliente real puede estar asociado a múltiples identificadores operativos (`customer_id`) en el sistema origen.
- Conservación de atributos relevantes como código postal, ciudad y estado.

Este proceso permite obtener una representación única de cada cliente real sin pérdida de información analítica.

2.3.3 Carga

Los clientes deduplicados se cargan en la tabla `dim_customer`, asignando una clave sustituta `customer_sk` a cada registro.

2.4 ETL de la dimensión producto (load_dim_producto.py)

2.4.1 Extracción

La información de productos se extrae del fichero `olist_products_dataset.csv`. Adicionalmente, se utiliza el fichero `producto_category_name_translation.csv` para enriquecer los datos con la traducción de las categorías de producto.

2.4.2 Transformación

Las principales transformaciones realizadas son:

- Normalización de nombres de columnas con errores tipográficos presentes en el origen.
- Limpieza y estandarización de campos de texto.
- LEFT JOIN con la tabla de traducción de categorías para obtener el nombre de categoría en inglés.
- Gestión explícita de valores nulos en aquellas categorías que no disponen de traducción.
- Deduplicación preventiva por `producto_id`.

2.4.3 Carga

Los productos transformados se cargan en la tabla `dim_product`, asignando una clave sustituta `producto_sk` y conservando tanto la categoría original como la traducida para análisis posteriores.

2.5 ETL de la tabla de hechos de ventas (`load_fact_sales.py`)

2.5.1 Extracción

La tabla de hechos de ventas se construye principalmente a partir del fichero `olist_order_items_dataset.csv`, que define el grano del hecho. Para completar la información se extraen datos adicionales de:

- `olist_orders_dataset.csv`
- `olist_customers_dataset.csv`

2.5.2 Transformación

Durante esta fase se realizan múltiples transformaciones:

- Uniones entre los distintos ficheros utilizando el identificador de pedido (`order_id`).
- Obtención del identificador lógico del cliente (`customer_unique_id`) y su traducción a `customer_sk`.
- Traducción del identificador del producto (`producto_id`) a `product_sk`.
- Generación de la clave temporal `date_sk` a partir de la fecha de compra del pedido.
- Conservación de métricas económicas (`price`, `freight_value`) y atributos de contexto del pedido.

La granularidad final es una fila por producto vendido dentro de un pedido.

2.5.3 Carga

Los registros resultantes se insertan en la tabla `fact_sales`, manteniendo las relaciones con las dimensiones mediante claves foráneas.

2.6 ETL de la tabla de hechos de pagos (`load_fact_payment.py`)

2.6.1 Extracción

Los datos de pagos se extraen del fichero `olist_order_payments_dataset.csv`. Para obtener información temporal se utilizan datos adicionales del fichero `olist_orders_dataset.csv`.

2.6.2 Transformación

Las transformaciones realizadas incluyen:

- Unión de pagos con pedidos para obtener la fecha de compra.
- Generación de la clave temporal date_sk.
- Conservación de atributos relevantes del pago, como método de pago, número de cuotas e importe.

El grano definido es una fila por registro de pago.

2.6.3 Carga

Los registros transformados se cargan en la tabla fact_payment, permitiendo análisis de pagos por tiempo y método.

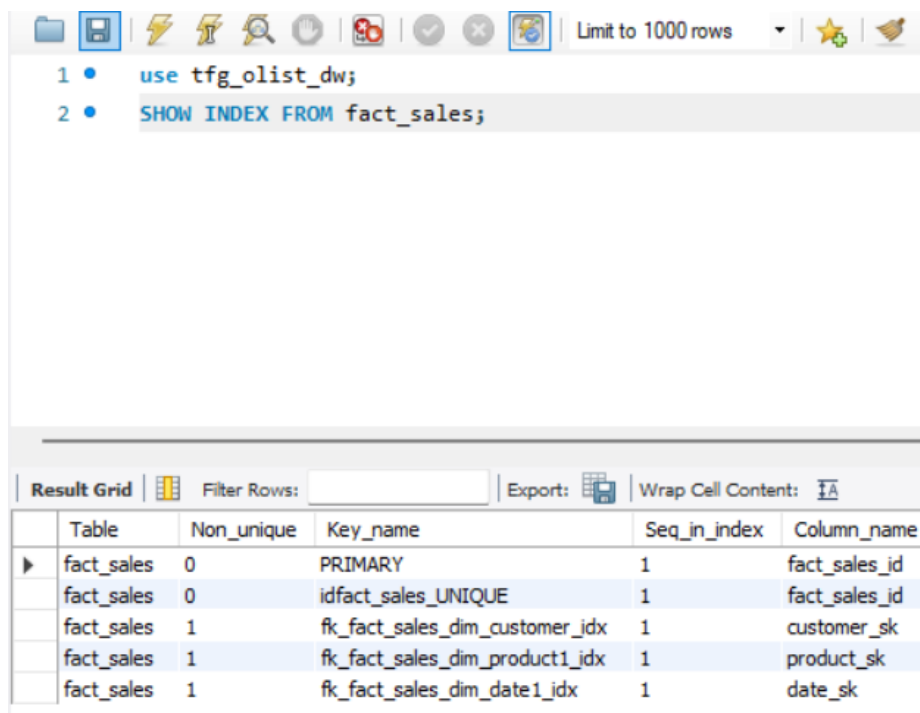
2.7 Pruebas tras el proceso ETL

Una vez completada la carga de datos, se realizaron pruebas para validar el sistema.

2.7.1 Pruebas de optimización

Se seleccionaron consultas analíticas representativas (agregaciones por tiempo, producto y cliente) y se analizó su plan de ejecución mediante EXPLAIN. Dado que el esquema sigue un modelo en estrella, se verificó que las claves foráneas de las tablas de hechos disponen de índices (generados automáticamente por MySQL al crear las restricciones). Los planes de ejecución confirmaron el uso de dichos índices en las operaciones de unión, evitando recorridos completos de las tablas de hechos.

- Índices generados.



The screenshot shows a MySQL command-line interface with the following commands and results:

```

1 • use tfg_olist_dw;
2 • SHOW INDEX FROM fact_sales;

```

The results are displayed in a table with the following columns: Table, Non_unique, Key_name, Seq_in_index, and Column_name.

Table	Non_unique	Key_name	Seq_in_index	Column_name
fact_sales	0	PRIMARY	1	fact_sales_id
fact_sales	0	idfact_sales_UNIQUE	1	fact_sales_id
fact_sales	1	fk_fact_sales_dim_customer_idx	1	customer_sk
fact_sales	1	fk_fact_sales_dim_product1_idx	1	product_sk
fact_sales	1	fk_fact_sales_dim_date1_idx	1	date_sk

Limit to 1000 rows

```

1 • use tfg_olist_dw;
2 • SHOW INDEX FROM fact_payment;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	Table	Non_unique	Key_name	Seq_in_index	Column_name
▶	fact_payment	0	PRIMARY	1	fact_payment_id
	fact_payment	1	fk_fact_payment_dim_date1_idx	1	date_sk

- Muestra de uso de índices con EXPLAIN (TABLA SALES Y PAYMENT).

Limit to 1000 rows

```

1 • EXPLAIN
2 SELECT d.year, d.month, SUM(f.price) AS total_sales
3 FROM fact_sales f
4 JOIN dim_date d ON f.date_sk = d.date_sk
5 WHERE d.year = 2018
6 GROUP BY d.year, d.month;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	id	select_type	table	partitions	type	possible_keys	key
▶	1	SIMPLE	d	NULL	ALL	PRIMARY	NULL
	1	SIMPLE	f	NULL	ref	fk_fact_sales_dim_date1_idx	fk_fact_sales_dim_date1_idx

2.7.2 Pruebas de integridad

Se realizaron pruebas de integridad referencial para validar que no existen claves huérfanas en las tablas de hechos. Para ello, se ejecutaron consultas LEFT JOIN desde las tablas de hecho hacia las dimensiones, confirmando que todas las claves sustitutas referencian registros existentes.

- fact_sales -> dim_customer.

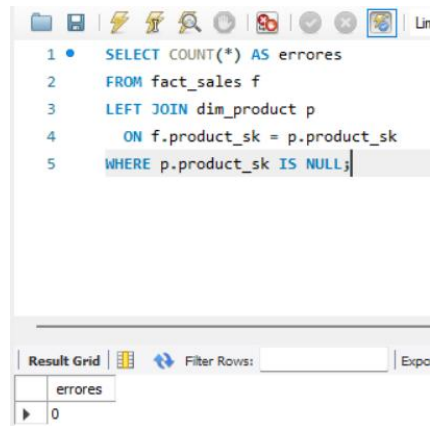
```

1 • SELECT COUNT(*) AS errores
2 FROM fact_sales f
3 LEFT JOIN dim_customer c
4     ON f.customer_sk = c.customer_sk
5 WHERE c.customer_sk IS NULL;

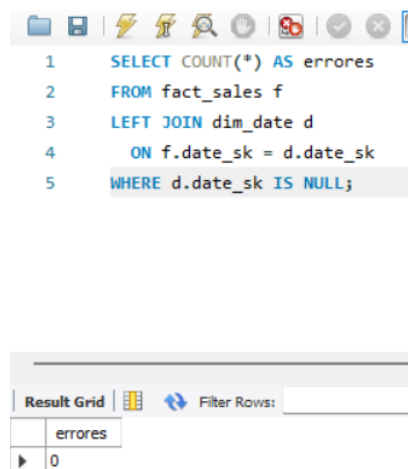
```

Result Grid	Filter Rows:	Export:
errores		
▶ 0		

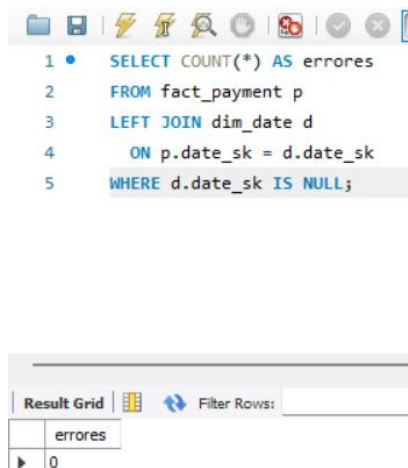
- fact_sales -> dim_product.



- fact_sales -> dim_date.



- fact_payment -> dim_date



2.7.3 Pruebas de rendimiento / estrés

Para evaluar el rendimiento del Data Warehouse se ejecutaron consultas analíticas representativas, incluyendo agregaciones simples y consultas con múltiples uniones entre la tabla de hechos y dimensiones. Se midieron los

tiempos de ejecución en MySQL Workbench y se repitió la consulta mas exigente varias veces para comprobar estabilidad y comportamiento con caché. Los tiempos obtenidos fueron adecuados para el volumen de datos cargado, confirmando que el sistema es estable y apto para su explotación mediante dashboards.

- Consulta simple + tiempo(SELECT SUM(price) FROM fact_sales;).

```
SELECT SUM(PRICE) as TOTAL FROM fact_sales LIMIT 0, 1000
```

1 row(s) returned

0.110 sec / 0.000 sec

- Consulta compleja + tiempo.

```
SELECT d.year, d.month, p.product_category_name_english,  
SUM(f.price) AS total_sales  
FROM fact_sales f JOIN dim_date d ON f.date_sk = d.date_sk JOIN  
dim_product p ON f.product_sk = p.product_sk  
GROUP BY d.year, d.month, p.product_category_name_english  
ORDER BY d.year, d.month, total_sales DESC;
```

```
SELECT d.year, d.month, p.product_category_name_english, SUM(f.price) AS total_sales FROM fact_sal... 1000 row(s) returned
```

0.438 sec /

- Consulta compleja repetida + tiempo para observar caché.

```
19:41:45 SELECT d.year, d.month, p.product_category_name_english, SUM(f.price) AS total_sales FROM fact_sal... 1000 row(s) returned
```

0.360 sec /

```
19:41:47 SELECT d.year, d.month, p.product_category_name_english, SUM(f.price) AS total_sales FROM fact_sal... 1000 row(s) returned
```

0.343 sec /

```
19:41:49 SELECT d.year, d.month, p.product_category_name_english, SUM(f.price) AS total_sales FROM fact_sal... 1000 row(s) returned
```

0.438 sec /